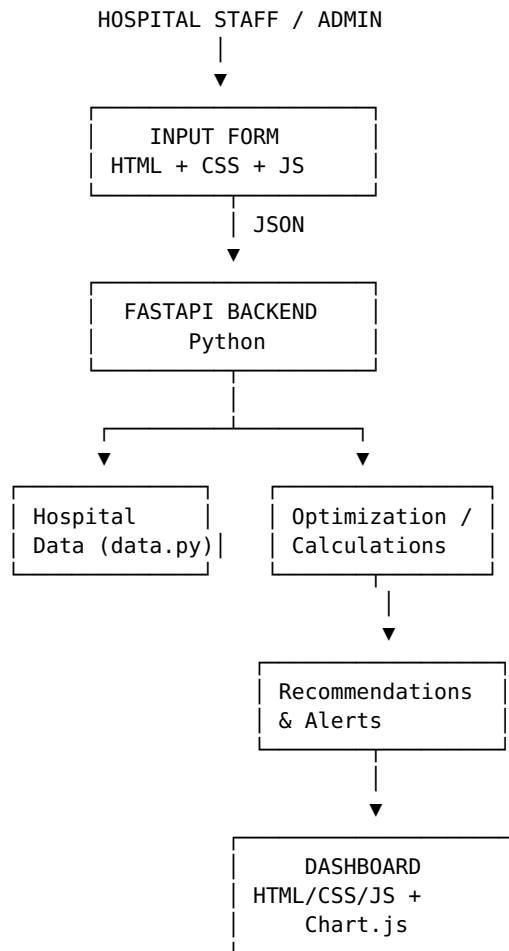


ARCHITECTURE

System Architecture

High-Level Overview



Layers

1. Frontend (index.html, login.html)

- Renders the dashboard: hospital overview, bed cards, doctor/nurse panels, equipment, patients, alerts, and the utilization chart.
- Reads form input values via the DOM (e.g. `document.getElementById("inputICU0occupied").value`).
- Converts collected values into a JSON payload.
- Sends data to the backend with the Fetch API and re-renders the page with the response.

2. Communication Layer

- **Protocol:** REST over HTTP, JSON payloads.

- **CORS:** Configured via `CORSMiddleware` so a frontend served separately (e.g. Live Server on 127.0.0.1:5500) can call the FastAPI backend on 127.0.0.1:8000.

3. Backend (main.py)

- Built with **FastAPI**, run via **Uvicorn** (`py -m uvicorn main:app --reload`).
- Validates incoming data using **Pydantic** models, e.g.:

```
class HospitalUpdate(BaseModel):
    icu_total: int
    icu_occupied: int
```

- Exposes REST endpoints (`/status`, `/recommendations`, `/simulate/normal`, `/simulate/surge`, `/update`).
- Documented automatically via Swagger/OpenAPI at `/docs`.

4. Data Layer (data.py)

- Holds hospital data as an **in-memory Python data structure** (dictionaries) — beds, doctors, nurses, equipment, patients.
- Updated in place when `/update` is called.
- Reset to defaults whenever the server restarts (current prototype limitation — see README).

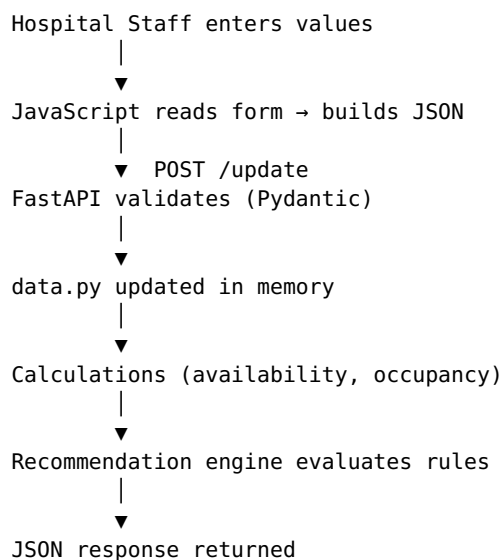
5. Optimization / Calculation Layer

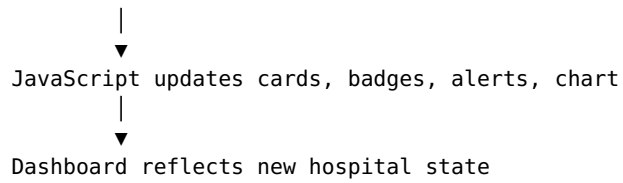
- Pure functions/logic inside the backend that compute:
 - Available beds = Total – Occupied
 - Occupancy % = (Occupied / Total) × 100
 - Busy staff = Total – Available
 - Equipment in use = Total – Available

6. Recommendation Engine

- Rule-based thresholds applied to the calculated values to classify status as `NORMAL`, `WARNING`, or `CRITICAL`, and to generate human-readable alert messages.

Data Flow (End-to-End)





Why FastAPI + Vanilla JS?

- **FastAPI** gives fast development, automatic request validation (Pydantic), and free interactive API docs (Swagger) — ideal for rapid hackathon iteration.
- **Vanilla HTML/CSS/JS** keeps the frontend lightweight and dependency-free, with Chart.js added only for visualization.

Future Architecture (Planned Extension)

